

TRABAJO PRÁCTICO NRO 4
SEMINARIO DE PRÁCTICA
DE INFORMATICA

Sistema de Gestión Integral
para una Clínica de Salud

Antonella Diaz
DNI 28.910.424
VINFOR12606

<https://github.com/antonellaanahi/SEMINARIODEPRACTICA>

Índice

1. Selección del patrón de diseño y justificación.
2. Persistencia y consulta de datos en una base de datos MySQL. Esta actividad incluye establecer conexiones, realizar consultas, actualizar registros y presentar resultados en la interfaz.
3. Correcta aplicación de excepciones para la interacción con la base de datos MySQL.
4. Inclusión pertinente de clases abstractas o interfaces.
5. Utilización complementaria de arreglos y de la clase ArrayList

Consideración:

La Clínica de Salud requiere un sistema que permita administrar de manera centralizada la información de pacientes, médicos, turnos, inventario y facturación. Actualmente estos procesos pueden realizarse mediante registros manuales o utilizando diferentes aplicaciones independientes, lo que genera demoras, duplicación de información y una mayor posibilidad de cometer errores.

El proyecto propone desarrollar un Sistema de Gestión Integral que reúna toda la información en una única plataforma. De esta manera se optimizan las tareas administrativas, se mejora la organización de los datos y se brinda una atención más eficiente a los pacientes.

Los principales usuarios del sistema serán el personal de recepción, los médicos, el área administrativa y los responsables del control del inventario.

Proceso Unificado de Desarrollo:

El desarrollo del proyecto se realizó siguiendo las etapas del Proceso Unificado de Desarrollo (PUD), permitiendo organizar el trabajo de forma iterativa e incremental.

Inicio: se identificó la problemática de la clínica y se definieron los objetivos del sistema.

Elaboración: se analizaron los requerimientos, se diseñó la arquitectura y se seleccionó el patrón DAO para el acceso a los datos.

Construcción: se desarrolló el prototipo utilizando Java y una base de datos MySQL, implementando la persistencia de la información y el manejo de excepciones.

Transición: se realizaron pruebas de funcionamiento para verificar las operaciones de alta, baja, modificación y consulta de los datos.

Desarrollo:

Para continuar con el desarrollo del Sistema de Gestión Integral para Clínicas de Salud, vamos a incluir una selección de patrón de diseño, persistencia y consulta de datos en una base de datos MySQL, correcta aplicación de excepciones para la interacción con la base de datos, inclusión de clases abstractas o interfaces y utilización de arreglos y la clase ArrayList.

1. Selección del Patrón de Diseño y Justificación

Patrón de Diseño: DAO (Data Access Object)

Justificación: Se seleccionó el patrón DAO (Data Access Object) porque permite separar la lógica de acceso a los datos de la lógica de negocio del sistema. Gracias a esta arquitectura, cualquier modificación en la base de datos puede realizarse sin afectar el funcionamiento del resto de la aplicación. Además, facilita el mantenimiento del código, mejora la reutilización de componentes y permite escalar el sistema de manera más sencilla a medida que se incorporen nuevos módulos.

2. Persistencia y Consulta de Datos en una Base de Datos MySQL

Configuración del Entorno

Primero, necesitamos agregar la biblioteca de MySQL Connector a nuestro proyecto. Esto se puede hacer descargando el archivo JAR del conector MySQL y añadiéndolo al classpath del proyecto.

Dependencia Maven :

En XML

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.27</version>
</dependency>
```

Configuración de la Base de Datos MySQL

Creamos una base de datos MySQL llamada clinica y las tablas necesarias para almacenar la información de pacientes, médicos, citas, inventarios y facturación.

En SQL

Para almacenar la información del sistema se diseñó una base de datos relacional denominada ClinicaDB, compuesta por 8 tablas que representan las principales entidades del negocio. Se incorporaron tablas adicionales como Historias_Clinicas, Proveedores y Reportes para cubrir todas las funcionalidades del sistema. La estructura mantiene la integridad de los datos mediante claves primarias y claves foráneas.

```
CREATE DATABASE IF NOT EXISTS ClinicaDB;
USE ClinicaDB;

CREATE TABLE Pacientes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50),
  apellido VARCHAR(50),
  fecha_nacimiento DATE,
  direccion VARCHAR(100),
  telefono VARCHAR(20),
  correo_electronico VARCHAR(50)
);

CREATE TABLE Medicos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50),
  apellido VARCHAR(50),
  especialidad VARCHAR(50),
  telefono VARCHAR(20),
  correo_electronico VARCHAR(50)
);

CREATE TABLE Proveedores (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100),
  contacto VARCHAR(100)
);

CREATE TABLE Citas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  paciente_id INT,
  medico_id INT,
  fecha_hora DATETIME,
  estado VARCHAR(20),
  FOREIGN KEY (paciente_id) REFERENCES Pacientes(id),
  FOREIGN KEY (medico_id) REFERENCES Medicos(id)
);

CREATE TABLE Historias_Clinicas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  paciente_id INT,
  medico_id INT,
```

```

    fecha DATE,
    detalles TEXT,
    FOREIGN KEY (paciente_id) REFERENCES Pacientes(id),
    FOREIGN KEY (medico_id) REFERENCES Medicos(id)
);

CREATE TABLE Inventarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    producto VARCHAR(100),
    cantidad INT,
    fecha_vencimiento DATE,
    proveedor_id INT,
    FOREIGN KEY (proveedor_id) REFERENCES Proveedores(id)
);

CREATE TABLE Facturas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    paciente_id INT,
    fecha DATE,
    total DECIMAL(10, 2),
    estado VARCHAR(20),
    FOREIGN KEY (paciente_id) REFERENCES Pacientes(id)
);

CREATE TABLE Reportes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    tipo_reporte VARCHAR(50),
    fecha_creacion DATE,
    detalles TEXT
);

```

Código Java para Conexión y Manejo de Base de Datos

Clase de Conexión a la Base de Datos:

La conexión entre la aplicación desarrollada en Java y la base de datos MySQL se realiza mediante JDBC. Para ello se implementó la clase ConexionDB que establece la conexión utilizando el driver MySQL Connector/J 9.1.0, descargado directamente desde Maven Central y añadido al classpath del proyecto.

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

```

```

public class ConexionDB {
    private static final String URL =
        "jdbc:mysql://localhost:3306/clinicadb?useSSL=false&serverTimezone=UTC";
    private static final String USER = "root";
    private static final String PASSWORD = "123456";

    public static Connection getConexion() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}

```

DAO Interface y Clase Abstracta:

Se implementó la clase SistemaGestionClinica que centraliza toda la lógica del sistema a través de un menú interactivo con 8 opciones: gestión de pacientes, médicos, citas, historias clínicas, inventarios, facturación, reportes y salida. Cada operación obtiene una conexión mediante ConexionDB y utiliza PreparedStatement para ejecutar las consultas de forma segura, manejando los errores con bloques try-catch que capturan SQLException.

```

import java.util.List;

public interface DAO<T> {
    void insertar(T t) throws SQLException;
    T obtener(int id) throws SQLException;
    List<T> obtenerTodos() throws SQLException;
    void actualizar(T t) throws SQLException;
    void eliminar(int id) throws SQLException;
}

public abstract class AbstractDAO<T> implements DAO<T> {
    protected Connection connection;

    public AbstractDAO() {
        try {
            this.connection = DatabaseConnection.getConnection();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

DAO de Paciente:

La clase PacienteDAO implementa los métodos definidos en la interfaz para administrar la información de los pacientes. En ella se desarrollan las operaciones de inserción, consulta, modificación y eliminación utilizando consultas SQL preparadas (PreparedStatement), lo que además mejora la seguridad frente a ataques de inyección SQL.

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class PacienteDAO extends AbstractDAO<Paciente> {
    @Override
    public void insertar(Paciente paciente) throws SQLException {
        String query = "INSERT INTO Paciente (nombre, direccion, telefono,
numeroHistoriaClinica) VALUES (?, ?, ?, ?)";
        PreparedStatement ps = connection.prepareStatement(query);
        ps.setString(1, paciente.getNombre());
        ps.setString(2, paciente.getDireccion());
        ps.setString(3, paciente.getTelefono());
        ps.setString(4, paciente.getNumeroHistoriaClinica());
        ps.executeUpdate();
    }

    @Override
    public Paciente obtener(int id) throws SQLException {
        String query = "SELECT * FROM Paciente WHERE id = ?";
        PreparedStatement ps = connection.prepareStatement(query);
        ps.setInt(1, id);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            return new Paciente(
                rs.getString("nombre"),
                rs.getString("direccion"),
                rs.getString("telefono"),
                rs.getString("numeroHistoriaClinica")
            );
        }
        return null;
    }

    @Override
    public List<Paciente> obtenerTodos() throws SQLException {
        String query = "SELECT * FROM Paciente";
        PreparedStatement ps = connection.prepareStatement(query);
        ResultSet rs = ps.executeQuery();
        List<Paciente> pacientes = new ArrayList<>();
        while (rs.next()) {
            pacientes.add(new Paciente(
                rs.getString("nombre"),
                rs.getString("direccion"),
                rs.getString("telefono"),
                rs.getString("numeroHistoriaClinica")
            ));
        }
    }
}
```

```

    ));
    }
    return pacientes;
}

@Override
public void actualizar(Paciente paciente) throws SQLException {
    String query = "UPDATE Paciente SET nombre = ?, direccion = ?, telefono = ?,
numeroHistoriaClinica = ? WHERE id = ?";
    PreparedStatement ps = connection.prepareStatement(query);
    ps.setString(1, paciente.getNombre());
    ps.setString(2, paciente.getDireccion());
    ps.setString(3, paciente.getTelefono());
    ps.setString(4, paciente.getNumeroHistoriaClinica());
    ps.setInt(5, paciente.getId());
    ps.executeUpdate();
}

@Override
public void eliminar(int id) throws SQLException {
    String query = "DELETE FROM Paciente WHERE id = ?";
    PreparedStatement ps = connection.prepareStatement(query);
    ps.setInt(1, id);
    ps.executeUpdate();
}
}

```

3. Correcta Aplicación de Excepciones

Al interactuar con la base de datos, es crucial manejar adecuadamente las excepciones para asegurar la robustez del sistema. Utilizaremos bloques try-catch para manejar SQLException y otras posibles excepciones.

La utilización de bloques try-catch permite detectar errores durante la interacción con la base de datos sin interrumpir la ejecución de la aplicación. Esto facilita informar al usuario cuando ocurre un problema y simplifica las tareas de mantenimiento y depuración del sistema.

Ejemplo de Manejo de Excepciones en PacienteDAO:

```

public class PacienteDAO extends AbstractDAO<Paciente> {
    @Override
    public void insertar(Paciente paciente) {
        String query = "INSERT INTO Paciente (nombre, direccion, telefono,
numeroHistoriaClinica) VALUES (?, ?, ?, ?)";
        try (PreparedStatement ps = connection.prepareStatement(query)) {
            ps.setString(1, paciente.getNombre());
            ps.setString(2, paciente.getDireccion());
            ps.setString(3, paciente.getTelefono());
            ps.setString(4, paciente.getNumeroHistoriaClinica());
            ps.executeUpdate();
        }
    }
}

```

```

    } catch (SQLException e) {
        System.err.println("Error al insertar paciente: " + e.getMessage());
    }
}

@Override
public Paciente obtener(int id) {
    String query = "SELECT * FROM Paciente WHERE id = ?";
    try (PreparedStatement ps = connection.prepareStatement(query)) {
        ps.setInt(1, id);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            return new Paciente(
                rs.getString("nombre"),
                rs.getString("direccion"),
                rs.getString("telefono"),
                rs.getString("numeroHistoriaClinica")
            );
        }
    } catch (SQLException e) {
        System.err.println("Error al obtener paciente: " + e.getMessage());
    }
    return null;
}

// Implementación de otros métodos con manejo de excepciones similar...
}

```

4. Inclusión de Clases Abstractas o Interfaces

En el desarrollo del sistema se implementó una interfaz denominada DAO<>, que define las operaciones básicas de acceso a los datos (insertar, obtener, actualizar y eliminar). De esta manera, todas las clases encargadas de interactuar con la base de datos mantienen una estructura común y uniforme.

Asimismo, se creó una clase abstracta denominada AbstractDAO<>, cuya función es centralizar la conexión con la base de datos MySQL. Gracias a esta implementación, las clases específicas como PacienteDAO heredan la conexión y evitan duplicar código, favoreciendo la reutilización y el mantenimiento de la aplicación.

La utilización conjunta de interfaces y clases abstractas permite aplicar los principios de la Programación Orientada a Objetos, especialmente la abstracción, la reutilización del código y el polimorfismo. Además, facilita la incorporación de nuevas entidades al sistema, como MedicoDAO o FacturaDAO, manteniendo la misma estructura de desarrollo y respetando el patrón DAO seleccionado.

5. Utilización Complementaria de Arreglos y de la Clase ArrayList

Para administrar colecciones dinámicas de objetos se utilizó la clase ArrayList, ya que permite agregar y eliminar elementos durante la ejecución del programa. En cambio, los arreglos se emplean cuando la cantidad de elementos es conocida y permanece constante, como sucede en el cálculo de determinadas estadísticas.

Usaremos ArrayList para manejar listas de objetos dinámicamente, mientras que los arreglos pueden ser utilizados para operaciones específicas donde el tamaño es fijo.

Ejemplo de uso de ArrayList en Inventario:

```
public class Inventario {
    private ArrayList<Producto> productos;

    public Inventario() {
        productos = new ArrayList<>();
    }

    public void agregarProducto(Producto producto) {
        productos.add(producto);
    }

    public void eliminarProducto(Producto producto) {
        productos.remove(producto);
    }

    public Producto buscarProducto(String nombre) {
        for (Producto producto : productos) {
            if (producto.getNombre().equalsIgnoreCase(nombre)) {
                return producto;
            }
        }
        return null;
    }

    public void mostrarInventario() {
        for (Producto producto : productos) {
            System.out.println(producto);
        }
    }
}
```

Ejemplo de uso de Arreglos:

Uso de Arreglo para una Operación Específica:

```
public class EstadisticasInventario {
    public static double calcularPromedioCantidad(Producto[] productos) {
        int total = 0;
        for (Producto producto : productos) {
```

```
        total += producto.getCantidad();
    }
    return total / (double) productos.length;
}
}
```

Conclusión:

El desarrollo del Sistema de Gestión Integral para Clínicas de Salud permitió aplicar de forma práctica los contenidos trabajados durante el módulo. La implementación del patrón DAO mejoró la organización del código al separar la lógica de acceso a los datos de la lógica de negocio, mientras que la utilización de MySQL garantizó una adecuada persistencia de la información.

Asimismo, la incorporación de interfaces, clases abstractas, manejo de excepciones, arreglos y la clase ArrayList permitió desarrollar un prototipo más organizado, reutilizable y escalable. Como mejora futura, el sistema podría incorporar funcionalidades como la gestión de historias clínicas digitales, turnos en línea y generación de reportes estadísticos para optimizar aún más la administración de la clínica.